RAG Interview Preparation Notes

Core concepts, production practice, and interview questions

These notes synthesize the important ideas across RAG Modules 1-5. They move from the retrieval and generation basics to production design, evaluation, optimization, security, and multimodal systems. The final question bank is a separate interview-prep addition; it is not presented as material from the PDFs.

One minute explanation

Retrieval augmented generation (RAG) searches a trusted knowledge base for information relevant to a user question, then gives the selected evidence to an LLM as context. The LLM uses that context to compose a response, often with citations. RAG is useful when answers depend on private, recent, or specialized information that may be absent from model training. Retrieval can reduce unsupported answers, but it does not guarantee that the model will use evidence correctly.

The RAG workflow

Stage	What happens	Design focus
Prepare knowledge	Extract source content, preserve metadata, split it into chunks, and create sparse or dense representations.	Coverage, clean text, chunk boundaries, permissions, and source identifiers.
Retrieve	Interpret the query, apply eligible metadata filters, and search by keywords, meaning, or both.	Relevant results, useful top-k, and permission-aware filtering.
Rerank	Rescore a smaller candidate set with a stronger relevance model when needed.	Higher precision within an acceptable latency and cost budget.
Generate	Build a prompt from instructions, conversation history, retrieved evidence, and the current question.	Grounded claims, clear citations, and a safe response when evidence is missing.
Evaluate and improve	Measure retrieval, response quality, latency, cost, and user feedback; inspect traces and test changes.	Find whether a failure came from data, retrieval, prompting, generation, or operations.

Contents

- 1 RAG fundamentals and system tradeoffs
- 2 Retrieval methods and embeddings
- 3 Vector databases and approximate search
- 4 Document preparation and chunking
- 5 Query understanding and reranking
- 6 LLM grounding and response generation
- 7 Evaluation, monitoring, and observability
- 8 Production optimization
- 9 Security, failure modes, and multimodal RAG
- 10 Practical implementation checklist
- 11 Interview practice additions
- 12 Source map

1 RAG fundamentals and system tradeoffs

RAG combines an LLM with an external knowledge base. The retriever finds useful material; the generator reasons over the material and responds. For a simple question already covered by the model, retrieval may add little. Questions about current events, private company data, a specific codebase, or specialized documents need evidence to be collected first.

Why use RAG

- Bring private, recent, or domain-specific information into the model's context without expecting the model's training data to contain it.
- Update the knowledge base as source information changes instead of retraining the model for every knowledge update.
- Ground answers in retrieved passages and attach source references that make human verification easier.
- Adapt answers to a company's manuals, policies, products, codebase, or a user's permitted personal data.

Common applications include codebase question answering, company support chatbots, specialized legal or medical research, web search summaries based on live results, and personal assistants that retrieve permitted messages, calendar entries, contacts, or files.

What RAG does not guarantee

The retriever can miss useful documents or return irrelevant ones. The LLM can ignore retrieved facts, combine them incorrectly, add unsupported details, or invent citations. RAG reduces some knowledge-gap errors; it does not eliminate hallucinations. More retrieved text can also increase latency and cost, consume the context window, and distract the model.

Choice	If you retrieve too little	If you retrieve too much
Number of documents	Evidence may be missing; recall falls.	Irrelevant passages use tokens and may dilute useful context.
Chunk size	Local context or definitions may be cut off.	A chunk may cover several topics and match a query only loosely.
Answer detail	The response may omit needed explanation.	Long generations cost more and create more room for unsupported claims.

2 Retrieval methods and embeddings

A useful retriever balances exact lexical matches, semantic similarity, and hard constraints. No single approach is ideal for every query or corpus, so evaluate choices against examples from the intended workload.

Method	How it works	Strengths and limits
Keyword search	Scores documents for query terms; modern systems commonly use BM25.	Fast and interpretable; preserves exact names, codes, and phrases. Misses synonyms and paraphrases.
Semantic search	Embeds the query and documents, then ranks vectors by a similarity or distance measure.	Finds related meaning without exact word overlap. Can miss rare exact terms and depends on embedding quality.
Metadata filtering	Uses fields such as title, date, author, region, or access rights to include or exclude records.	Fast and precise for rigid rules; it does not understand content or rank documents, so it is not a search method by itself.

Method	How it works	Strengths and limits
Hybrid search	Combines keyword and semantic rankings, then filters or fuses results.	Useful when exact terms and conceptual matches both matter; weights and result cutoffs need evaluation.

Keyword scoring

A bag-of-words representation tracks which terms occur and how often while ignoring word order. The resulting vectors are sparse because most vocabulary terms do not appear in a given document. Term frequency (TF) rewards occurrences; normalizing by document length prevents longer documents from winning simply because they contain more words. Inverse document frequency (IDF) gives more weight to rarer terms and less to common filler words.

BM25 refines TF-IDF. It uses term-frequency saturation so repeated occurrences help less after a point, and it normalizes for document length less aggressively than basic TF-IDF. Its k1 parameter controls the effect of term frequency; b controls length normalization. The module gives typical k1 values around 1.2-2.0 and b from 0 (none) to 1 (full).

Embeddings and semantic similarity

An embedding model maps words, sentences, or documents to dense vectors. The axes do not have simple human meanings; the useful property is that semantically related text tends to form nearby clusters. Contrastive training uses positive pairs that should be close and negative pairs that should be farther apart, repeatedly updating the model based on their scores.

Measure	Interpretation	Practical note
Euclidean distance	Straight-line distance between vectors.	Smaller distance means closer points.
Cosine similarity	Similarity in vector direction, largely independent of magnitude.	Often used for semantic comparison; values range from −1 to 1.
Dot product	Projection-based similarity that can reflect both direction and vector magnitude.	Can be negative, zero, or positive; use the metric expected by the embedding model and index.

Embedding compatibility

Compare query and document vectors produced by the same embedding model. Switching models can change vector spaces, so re-embedding or another explicitly supported migration is needed before comparing the new query vectors with old index vectors.

Combining rankings

Reciprocal Rank Fusion (RRF) rewards documents that rank highly across multiple lists. A common form sums 1 / (k + rank) across lists. With a small k, a high position has a strong effect; a larger k softens the difference between positions. RRF uses rank positions rather than raw scores. A separate hybrid weight can shift emphasis toward semantic or keyword search; lower semantic weight can help when exact terms matter. Metadata rules should represent eligibility constraints such as a user's region or access rights.

3 Vector databases and approximate search

Brute-force k-nearest-neighbor (KNN) search embeds the query, computes its distance to every stored vector, sorts the results, and returns the closest items. Work grows with corpus size: a million-document index requires a million distance calculations for each exhaustive query. Approximate nearest-neighbor (ANN) methods use additional structures to search much faster, at the cost of no guarantee that they find the exact closest vectors.

Approach	Search pattern	Tradeoff
KNN	Compare the query to every vector and sort by distance.	Exact nearest results, but the work grows linearly with the number of vectors.
NSW graph	Navigate a proximity graph from a candidate, moving through neighboring vectors until no closer candidate is found.	Fast in practice, but a local path can miss the global closest point.
HNSW	Search a sparse upper graph first, then descend through denser layers for a more precise search.	Makes large early jumps and narrows the search; index construction costs computation and memory.

A vector database stores high-dimensional vectors and supports ANN search, often with HNSW indexes. Typical operations include creating a collection and its properties, loading documents in batches, generating dense vectors for semantic search and sparse vectors for keyword search, creating the index, handling ingestion errors, and running vector, BM25, hybrid, or filtered queries. Source text and metadata must remain linked to each vector so the retrieved chunk can be used and cited.

4 Document preparation and chunking

Indexing an entire book as one vector averages its topics into a weak representation and returns too much text for a focused answer. Chunking improves topical precision and keeps the context sent to the LLM manageable. The goal is to preserve enough local meaning while avoiding a chunk that spans too many ideas.

Strategy	Method	Advantages and costs
Fixed size	Split at a set character or token length.	Simple baseline; boundaries can break sentences or concepts.
Overlap	Repeat a small tail of one chunk at the start of the next.	Reduces lost context at boundaries; increases storage and duplicate retrieval.
Recursive or structural	Split first on paragraph or section boundaries, then smaller separators; use structures such as HTML headings or code functions.	Variable-size chunks can respect document structure.
Semantic	Embed sequential sentences and begin a new chunk when meaning changes beyond a chosen threshold.	Can create more coherent boundaries and improve retrieval; repeated embeddings add preprocessing cost.
LLM-based	Ask a language model to group content by concept.	Can keep concepts together; adds a preprocessing call and requires review.
Context-aware	Add a short description or surrounding context to each chunk during preprocessing.	Can improve retrieval and generation without adding query-time latency; costs extra preprocessing and storage.

Fixed-size splitting with overlap and recursive splitting are reasonable starting points. There is no universal chunk size: compare retrieval quality, context use, and latency on representative questions. Keep source metadata and location information with every chunk so that results can be traced to a page, section, or code location.

5 Query understanding and reranking

Query parsing options

Technique	Use	Cost to watch
Query rewriting	Rewrite an ambiguous or conversational prompt for search; clarify terms, add useful synonyms, and remove distractions.	Adds an LLM call and latency; evaluate whether retrieval improves.
Named entity recognition	Extract people, places, dates, organizations, or other entities to support targeted search or filters.	Entity labels and extraction errors can affect downstream retrieval.
HyDE	Generate a hypothetical ideal document, embed it, and search using that document representation.	Can improve matching but adds generation cost and latency; the hypothetical text is not evidence.

Bi-encoders, cross-encoders, and ColBERT

Approach	How it scores	Quality, speed, and storage
Bi-encoder	Embed query and documents separately; compare precomputed document vectors with ANN.	Fast and storage-efficient; a good default for first-stage semantic search.
Cross-encoder	Concatenate each query-document pair and predict a relevance score directly.	Usually stronger pairwise relevance, but cannot precompute query-specific scores; expensive across millions of documents.
ColBERT	Store token-level vectors and score each query token against its best matching document token, then combine the matches.	Keeps richer interactions with better scalability than a full cross-encoder, at a larger vector-storage cost.

Reranking applies a slower, stronger scorer only to a limited candidate list from a fast first-stage retriever. It improves the order of candidates before the LLM sees them. The course examples return a small set of the best results after reranking; the correct candidate and final top-k sizes depend on latency and quality goals.

6 LLM grounding and response generation

How a transformer supports generation

Transformers process token representations that combine semantic information with position. Attention lets each token use information from other tokens, while feed-forward layers refine those representations. The model then predicts a likely next token and repeats the process autoregressively until it reaches an end token or output limit. Encoder components are common in embedding models; most text-generating LLMs are decoder-based.

This ability to process context explains why an LLM can use retrieved text. It does not make the model a truth detector. A language model predicts probable continuations from learned patterns; plausible wording can still be false. Longer prompts and outputs increase processing cost, and every generated token requires further model computation.

Prompt structure for grounded answers

A chat prompt commonly contains system, user, and assistant messages. System instructions establish behavior and procedure; user messages provide questions; assistant messages preserve earlier responses. A RAG template can include system rules, useful conversation history, retrieved chunks with source IDs, and the current question. System instructions are included with every request, so keep them deliberate.

Grounding instruction example

Use only the retrieved documents to answer. Decide whether the documents support the requested fact. Cite sources as [DOC n]. If the evidence is missing, say that the available documents do not establish the answer.

Examples in the prompt can teach a response pattern: one example is often called one-shot learning and several examples few-shot learning. Retrieved examples can help match a style or task, but they consume context. Include only examples that improve the result. For long conversations, prune stale messages and include chunks relevant to the latest question. Large context windows help with deeper conversations but do not remove the need for efficient prompts.

Sampling controls

Control	Effect	Use carefully
Greedy decoding	Always chooses the highest-probability next token; deterministic.	Can sound generic or repeat a loop.
Temperature	Changes the sharpness of the token probability distribution.	Lower values favor predictable output; higher values allow more variation and can become incoherent.
Top-k	Samples only from the k most probable tokens.	Keeps a fixed candidate count.
Top-p	Samples from the smallest set of tokens whose cumulative probability reaches a threshold.	Candidate count changes with model confidence.
Repetition penalty	Reduces probability for tokens already used.	Can discourage loops and repetitive phrasing.
Logit bias	Raises or lowers probabilities for specific tokens.	Can constrain or encourage selected vocabulary, but may distort fluent generation.

For factual or code-oriented tasks, the course recommends low temperature and top-p settings. Creative tasks can explore higher values. Sampling choices tune style and variance; they do not verify facts.

Reasoning models may use additional reasoning tokens, which can improve accuracy while increasing latency and cost. The modules caution that example mixing and some prompting techniques may work less well with these models; clear goals, strict formats, and full relevant context are better starting points.

Hallucination and citation checks

Retrieved evidence can still be ignored or misused. Repeating the same request and comparing answers may expose contradictions, but self-consistency checks are costly and unreliable. Asking for citations can make verification easier, yet a model can invent citations. Sentence-level attribution methods such as ContextCite link claims to supporting retrieved documents; citation evaluation should still check the evidence directly.

The ALCE benchmark evaluates generated answers along separate dimensions: fluency, factual correctness, and citation quality. Keeping these measures distinct helps identify whether an answer is readable, true, and properly supported.

RAG and fine-tuning

RAG	Fine-tuning
Best when the model needs new, current, private, or source-grounded information.	Best when the model needs to adapt to a task, domain, or response behavior.

RAG	Fine-tuning
Injects retrieved information at inference time and can show its sources.	Uses labeled examples to update model parameters; it is not a reliable way to add frequently changing facts.
Knowledge can be refreshed by updating the knowledge base.	Can improve a focused behavior but may reduce performance outside the target domain.
Can pair with fine-tuning so the model learns how to use retrieved evidence.	Can pair with RAG when both task adaptation and current evidence are needed.

Choosing a model and benchmark

Compare candidate models using the properties that affect the application: model size and cost per input or output token, context window, time to first token, tokens per second, and training cutoff. Larger models may be more capable but cost more; a larger context window does not remove per-token cost.

Model benchmarks can be automated, human-rated, or judged by another LLM. Automated tests are repeatable but may miss nuance. Human pairwise ratings capture preferences but cost more. LLM judges are flexible and cheaper to scale, but can favor their own model family. Select benchmarks that match the task, distinguish stronger from weaker candidates, reproduce reliably, align with real use, and have limited data contamination. Saturated tests stop separating models and need replacement.

Agentic RAG workflows

For complex tasks, separate stages such as routing, query parsing, retrieval, evaluation, response writing, and citation generation. A workflow can run these steps sequentially, branch conditionally after a router, iterate between a writer and evaluator, or run several agents in parallel before a synthesizer combines the results. Lightweight models can handle routing or evaluation while larger models draft the final answer. Each extra call adds latency and cost, so use a workflow only when its stages improve measured outcomes.

7 Evaluation, monitoring, and observability

Evaluate the retriever and generator separately as well as the end-to-end system. A retriever needs prompts, ranked results, and ground-truth labels showing which documents are relevant. Without ground truth, a metric cannot reliably tell whether a ranking change improved retrieval.

Retriever metrics

Metric	Definition	What it reveals
Precision@k	Relevant retrieved documents divided by all retrieved documents in the top k.	How much irrelevant context enters the prompt.
Recall@k	Relevant retrieved documents divided by all known relevant documents.	How much useful evidence the retriever leaves out.
Average precision / MAP@k	Average of precision values at ranks where relevant documents appear; MAP averages across prompts.	Whether relevant documents appear early in the ranked list.
Reciprocal rank / MRR	1 divided by the rank of the first relevant result; MRR averages over prompts.	How early the first useful result appears.

Metrics depend on k. A stricter top-5 view may show quality at the prompt boundary; recall@10 or a similar measure helps track whether enough relevant documents are present. Precision penalizes irrelevant results, while recall penalizes missing relevant results.

Generation and citation metrics

Measure	Question answered	Important limitation
Response relevancy	Does the answer address the user's prompt?	It can score a relevant but factually wrong answer well.
Faithfulness	Are factual claims supported by retrieved evidence?	Often uses an evaluator LLM to identify claims and judge support.
Citation quality	Do citations point to sources that support the associated claims?	A model can generate plausible-looking but false citations.
Fluency and correctness	Is the response readable and factually accurate?	These capture different properties and should not be collapsed into a single score.

LLM-as-a-judge evaluation is flexible and cheaper than reviewing every response by hand, but it needs a clear rubric and can be biased toward the judge's own model family. Human review captures nuance but costs more. Code checks are cheap and useful for measurable conditions such as JSON validity, token rates, or latency. Use a mix that fits each question.

Production monitoring

- Track system metrics such as end-to-end and component latency, throughput, memory, compute, tokens per second, and token usage.
- Record traces that connect the original prompt, rewritten query, retrieved chunks, metadata filters, reranker scores, final LLM prompt, response, citations, and latency.
- Keep prompts and responses as a useful baseline dataset; log additional component data when detailed diagnosis is needed. Data volume can grow quickly, so choose fields that support an evaluation decision.
- Use aggregate reports to find trends, component traces to locate causes, user feedback to reveal quality issues, and A/B tests to compare controlled changes.
- Separate system-level outcomes from component metrics. End-to-end latency shows that a problem exists; retriever-only or generation-only timing helps locate it.

A useful improvement loop is observe real traffic, identify a failure, change one part of the system, run an evaluation or A/B test, and check for regressions. The course examples include topic-level response quality analysis, context precision, faithfulness, and chunk counts to find that a router was sending diagram requests to the wrong component.

Custom evaluation datasets can store the original prompts, responses, and the path each request took through the system. What you retain determines which evaluations are possible. Clustering or classifying prompts by topic can expose quality differences between groups that aggregate averages hide. Use conventional infrastructure monitoring alongside LLM-focused observability for operational metrics.

8 Production optimization

Production design balances response quality, latency, and cost. More traffic raises compute, memory, and latency requirements; unpredictable prompts and higher failure stakes make monitoring part of system design rather than an afterthought.

Cost levers

The main cost areas are vector-database storage and queries, plus LLM inference and generation. Measure both instead of optimizing only one side of the pipeline.

Area	Options supported by the modules	Tradeoff to measure
LLM inference	Use a smaller model, quantize a model, use lightweight models for routing or evaluation, or fine-tune a small model for a narrow task.	Lower cost and memory can reduce response quality; compare on the target workload.
Prompt and output	Retrieve fewer chunks, remove unnecessary prompt content, request concise responses, or set an output token limit.	Less context can omit evidence; shorter output can omit explanation.
Hosting	Use a dedicated GPU endpoint when predictable hourly capacity is preferable to per-token pricing.	Compare utilization and hourly cost with request-based pricing.
Vector storage	Keep frequently used HNSW indexes in RAM, move less-used vectors to SSD or disk, and keep document bodies in lower-cost object storage.	Lower-cost tiers can increase access time.
Tenant data	Partition tenant indexes and load active data into faster memory when needed.	More operational management; authorization must remain correct.

Quantization and vector size

Quantization stores values at lower precision to reduce memory, storage, and compute. The modules describe compressing typical 16-bit LLM parameters to 8-bit or 4-bit equivalents and converting 32-bit embedding floats to 8-bit integers using a scale and offset. Eight-bit quantization can retain most retrieval quality in the cited course examples; very low-bit or 1-bit representations can have noticeable recall loss. A fast low-precision first pass followed by full-precision reranking is one way to balance speed and relevance.

Matryoshka-style embeddings place more distinguishing information in earlier dimensions, allowing a short vector for a quick first search and a larger vector for precise reranking. Check the embedding model's support before relying on this property.

Latency levers

- Measure the whole pipeline first. The course identifies LLM calls and transformer generation as the main latency bottleneck in its examples; retrieval and database steps are often faster.
- Try smaller or quantized models, smaller prompts, and a router that skips unnecessary steps. Keep only query rewriting, reranking, or other components that show measurable benefit.
- Cache exact or sufficiently similar prompts when responses remain valid. A personalized cache can pass a cached answer and the new prompt to a small model for adjustment.
- Use quantized embeddings or database sharding when retrieval itself needs to scale. Measure recall and latency after changing the index.
- For tenant data with predictable activity, move active indexes into faster storage when needed and keep inactive data in lower-cost tiers; time-zone patterns can help decide when a tenant is likely to be active.
- Set latency goals by use case. A customer service workflow may place a high priority on speed; a rare-disease research workflow may place more weight on answer quality.

9 Security, failure modes, and multimodal RAG

Security and privacy

- Authenticate users before retrieval and ensure each person can access only authorized information. Tenant separation can use distinct databases or one database with strict metadata filters.

- Treat retrieved private data as exposed to the LLM endpoint. The course notes that a RAG system can run locally on premises when data must remain inside the deployment boundary.
- Vector databases can be attacked like other databases. ANN search needs vectors in memory; encrypt text chunks where appropriate and account for the added decryption and latency complexity.
- Dense vectors may allow reconstruction of source text. The module lists adding noise, transforming vectors, and reducing dimensionality while preserving useful distances as possible mitigations.
- Log enough to investigate access and quality failures while controlling retention of sensitive prompts, retrieved text, and responses.

A production failure may be a confidently invented discount, a wrong answer that sounds plausible, a user request that seeks unauthorized facts, or a retrieval path that includes another tenant's data. Malicious users may try to obtain free products or secrets. Validate the output and access path; do not assume that a successful retrieval or a citation makes the answer safe.

Common failure modes

Symptom	Likely area to inspect
Answer misses a fact present in the corpus	Extraction quality, metadata, chunk boundary, query wording, embedding match, top-k, or reranking.
Answer cites a source that does not support it	Citation generation and validation; compare claim-level evidence with the cited chunk.
Answer is plausible but unsupported	Prompt rules, irrelevant context, model behavior, and faithfulness evaluation.
Unexpectedly slow requests	Per-stage traces, LLM calls, prompt length, reranker, repeated components, or index scaling.
Sensitive data appears for the wrong user	Authentication, tenant isolation, filter logic, and the full retrieval-to-prompt path.
Quality drops after a change	Run a stable evaluation set and compare component and system metrics before and after.

Multimodal RAG

Multimodal RAG extends retrieval to images, PDFs, slide decks, audio, or video. Both the retriever and the generation model need compatible multimodal capabilities. A multimodal embedding model can place text and images in a shared representation so a text query can retrieve an image or an image can be matched with text.

Vision-language models process text and image tokens together. The modules describe image tokenization into patches, often hundreds of tokens, followed by a transformer that learns relationships between the modalities. For dense PDFs and slides, patch-based retrieval can retain charts, captions, and visual layout that text extraction misses. One approach splits a page into image patches, embeds them, and scores them with ColBERT-like token-to-patch maximum similarities. It is flexible, but produces many vectors and increases index storage.

10 Practical implementation checklist

A workable build sequence follows the dependencies in the five modules. Treat the checklist as a starting workflow; verify each choice against your data and evaluation set.

1. Inventory the knowledge sources and access rules. Identify unstructured documents, tables, code, PDFs, slides, images, and missing metadata.
2. Extract readable content and preserve source, page or section, date, author, tenant, and permission metadata where available.
3. Choose a baseline chunker such as fixed-size splitting with overlap or recursive splitting. Preserve location metadata and compare alternatives on representative queries.

4. Create the vector database collection, load data in batches, generate dense embeddings for semantic search and sparse representations for keyword search, and build an ANN index.
5. Implement strict metadata filters for user eligibility. Test that unauthorized records never reach the prompt.
6. Start with keyword, semantic, or hybrid retrieval based on query needs. Set a candidate count and final top-k, then measure recall and precision.
7. Add query rewriting, entity extraction, HyDE, or reranking only when an evaluation shows benefit relative to added cost and latency.
8. Use a prompt template with clear system rules, relevant history, identified retrieved sources, and the current question. Instruct the model to cite evidence and say when it is missing.
9. Evaluate retriever rankings with labeled data. Evaluate response relevancy, faithfulness, citation support, correctness, and fluency separately.
10. Instrument end-to-end traces and component metrics. Capture the query, retrieved chunks, transformations, final prompt, response, latency, token use, and feedback needed for diagnosis.
11. Run controlled experiments. Change one component at a time, compare against a stable set, and monitor real traffic for regressions.
12. Tune cost and latency using measured bottlenecks. Keep a quality check after reducing model size, context, precision, or index work.
13. For multimodal sources, check extraction quality and whether both retrieval and generation support the required media type.

11 Interview practice additions

How to read this section

The following questions and concise model answers are added for interview practice. They are not transcripts of interview questions from the PDFs. The answers use the course concepts and frame them in the style of an AI Engineer discussion.

Fundamentals

1. What is retrieval augmented generation?

Model answer RAG retrieves relevant evidence from an external knowledge base and includes it in an LLM prompt. The model then generates an answer using that context, which is useful for current, private, or specialized facts.

2. Walk through a RAG request end to end.

Model answer The system receives a query, optionally rewrites it, applies access and metadata filters, retrieves candidates, optionally reranks them, builds a prompt with sources and instructions, generates an answer, and records traces and metrics.

3. When would you use RAG instead of fine-tuning?

Model answer Use RAG when the model needs new or changing facts, private data, or source citations. Fine-tuning is more suited to a task or response-style adaptation. They can be combined when the model should both behave consistently and use current evidence.

4. Does RAG eliminate hallucinations?

Model answer No. Relevant evidence can reduce knowledge-gap errors, but retrieval may fail and the model may ignore, misread, or extend beyond the evidence. I would evaluate faithfulness and citation support and define behavior for missing evidence.

5. Why not put the entire knowledge base in the prompt?

Model answer It would exceed practical context limits, increase per-token computation and cost, and make relevant evidence harder to isolate. Retrieval selects a smaller context for each question.

6. What should the model do when no retrieved document answers the question?

Model answer The prompt should direct it to say that the available evidence is insufficient instead of inventing a fact. I would test this case explicitly and verify that the citation behavior does not imply unsupported certainty.

Retrieval and indexing

7. How do sparse and dense retrieval differ?

Model answer Sparse keyword retrieval scores overlap in terms and is strong for exact names, identifiers, and phrases. Dense retrieval compares embeddings and can find paraphrases or related meanings without exact word overlap.

8. How does BM25 improve on basic TF-IDF?

Model answer BM25 adds term-frequency saturation and a tunable document-length normalization. This reduces the benefit of repeating a term indefinitely and avoids penalizing long documents as aggressively as basic normalization.

9. What is hybrid search, and how can you combine rankings?

Model answer Hybrid search combines lexical and semantic results, often alongside metadata constraints. Reciprocal Rank Fusion combines rank positions so a document that appears high on multiple lists is rewarded; weights can shift emphasis toward exact keywords or semantic matches.

10. How are metadata filters different from retrieval?

Model answer A filter applies rigid eligibility rules such as date, region, or access rights. It does not understand content or rank relevance, so it narrows the eligible set while keyword or semantic search finds matching content.

11. What does an embedding represent, and how is it learned?

Model answer It is a dense vector representation of text. Contrastive training pulls positive pairs closer and pushes negative pairs apart, shaping a space where related texts tend to be near each other.

12. How do cosine similarity, dot product, and Euclidean distance differ?

Model answer Euclidean distance measures straight-line distance. Cosine compares direction, while dot product also reflects vector magnitude. The index should use the scoring convention expected by the embedding model.

13. Why should query and document embeddings come from the same model?

Model answer Each model defines its own vector space. A query vector from a different model may not be comparable to the indexed document vectors, even if their dimensions happen to match.

14. Why use ANN instead of brute-force KNN?

Model answer KNN compares the query with every vector and scales linearly with the corpus. ANN uses an index such as a proximity graph to search much faster, accepting that it may not return the exact nearest neighbor.

15. Explain HNSW at a high level.

Model answer HNSW builds layered proximity graphs. Search begins in a sparse upper layer to make larger jumps, then descends into denser layers to refine candidates. This speeds search at the cost of index-building work and approximate results.

16. What role does a vector database play in a RAG system?

Model answer It stores vectors and associated content or metadata, builds an ANN index, and supports vector, keyword, hybrid, and filtered retrieval. It also supports ingestion operations that need batching and failure handling.

17. Why chunk documents, and how would you select a chunking strategy?

Model answer Chunking gives the index more focused representations and limits how much irrelevant text reaches the LLM. I would start with fixed-size overlap or recursive structural splitting, then compare semantic or context-aware methods using retrieval metrics and prompt cost.

18. What does overlap solve, and what does it cost?

Model answer Overlap reduces the chance that a key phrase or sentence is split at a boundary and gives boundary text surrounding context. It increases index size and may return duplicate content.

19. What is reranking?

Model answer A fast retriever returns a candidate set, and a stronger but slower model rescales those query-document pairs. This improves ordering without applying the expensive scorer to the entire corpus.

20. Compare bi-encoders, cross-encoders, and ColBERT.

Model answer Bi-encoders precompute one vector per document and are fast. Cross-encoders jointly score each query-document pair and often rank more accurately but cost more. ColBERT stores token vectors and uses late interaction, trading more storage for richer matching.

21. When might you rewrite a query or use HyDE?

Model answer Rewriting can clarify conversational phrasing or add useful terminology; entity extraction can support precise filters. HyDE generates a hypothetical relevant document and uses its embedding for search. I would keep these only if measured retrieval gains justify their added latency and cost.

Evaluation and generation

22. Explain precision and recall for retrieval.

Model answer Precision@k is the relevant portion of the retrieved top-k; recall@k is the portion of all known relevant documents that appear in that top-k. Precision penalizes irrelevant context, while recall penalizes missed evidence.

23. What do MAP and MRR tell you?

Model answer MAP rewards systems that place relevant documents early across a set of queries. MRR focuses on the rank of the first relevant document, which is useful when the first good result matters most.

24. How does top-k affect RAG quality?

Model answer A larger k can improve recall but also adds irrelevant or redundant context, tokens, latency, and cost. I would choose it using both retrieval metrics and downstream answer quality.

25. What is the difference between response relevancy and faithfulness?

Model answer Response relevancy asks whether the answer addresses the question. Faithfulness asks whether its factual claims are supported by retrieved evidence. An answer can be relevant but unfaithful.

26. How would you evaluate citations?

Model answer Check whether each factual claim is linked to a source that actually supports it. Citation quality is distinct from fluency and answer correctness; a generated citation can itself be false.

27. What are the tradeoffs of LLM-as-a-judge evaluation?

Model answer It is flexible and cheaper than broad human review, but needs a clear rubric and can have model-family bias. I would calibrate it against human-labeled examples and use code checks for deterministic requirements.

28. How would you debug a wrong answer?

Model answer Trace the request through query rewriting, filters, retrieved chunks, reranking, final prompt, and response. Then identify whether the cause is missing or poor source data, retrieval, prompt assembly, generation, or access control, and add a regression case.

Production and design

29. What would you log for a production RAG system?

Model answer At minimum, capture the query, transformations, filters, retrieved chunks, reranking, final LLM prompt, answer, citations, and per-stage latency. Track aggregate latency, throughput, token use, memory, compute, and user feedback while controlling sensitive-data retention.

30. How would you reduce latency without guessing?

Model answer Measure per-stage traces first. If generation dominates, try smaller or quantized models, shorter prompts, fewer retrieved chunks, caching, or skipping components that show no benefit. Recheck quality after each change.

31. How would you reduce inference and storage costs?

Model answer Use smaller or quantized models where quality remains acceptable, reduce unnecessary prompt and output tokens, and tier index storage so frequently used HNSW data stays fast while colder data moves to lower-cost storage. Compare the total quality and latency impact.

32. How do you protect multi-tenant RAG data?

Model answer Authenticate the user and enforce authorization before retrieved content enters the prompt. Use separate tenant indexes or strict metadata filtering, then test the entire retrieval path for cross-tenant leakage.

33. Can embeddings expose source text?

Model answer The course notes that reconstruction from dense vectors is possible. Mitigations it lists include adding noise, transforming vectors, or reducing dimensions while preserving useful distances; data protection also includes securing the underlying database and text chunks.

34. How would you handle PDFs and slides with charts and images?

Model answer Use extraction or vision-based ingestion that captures text, captions, charts, and images. A patch-based multimodal retriever can preserve visual content, but it creates many vectors, so storage and retrieval costs need evaluation.

35. What changes for multimodal RAG?

Model answer Both retrieval and generation need the required modality support. Shared multimodal embeddings can match text with images; a vision-language model can process image and text tokens together.

36. What is agentic RAG, and when is it useful?

Model answer Agentic RAG breaks a complex request into workflow steps such as routing, research, evaluation, and citation. It is useful when those steps improve the task, but each extra call can add latency and cost, so simpler workflows should remain simple.

37. How do temperature, top-k, and top-p relate?

Model answer Temperature changes distribution sharpness. Top-k limits sampling to a fixed number of likely tokens; top-p keeps a probability-mass-based set that changes with confidence. For factual tasks, lower sampling randomness is a reasonable starting point, followed by evaluation.

38. How would you combine RAG and fine-tuning?

Model answer Use RAG to supply current evidence and fine-tuning to teach a focused response behavior or improve how a model uses retrieved evidence. Evaluate both task quality and performance outside the target domain.

12 Source map

The core notes synthesize the five course PDFs. This map points to the module most relevant to each subject; page numbers are PDF page numbers.

Source	Main topics used in these notes
Module 1, PDF pp. 5-53	RAG purpose and architecture, use cases, knowledge gaps, retrieval versus generation, grounding, hallucination risks, context limits, and retriever tradeoffs.
Module 2, PDF pp. 5-85	Keyword and semantic search, metadata filters, TF-IDF, BM25, embeddings, similarity measures, hybrid search, RRF, precision, recall, MAP, and MRR.
Module 3, PDF pp. 2-82	KNN, ANN, NSW and HNSW, vector databases, ingestion, chunking, semantic and context-aware splitting, query rewriting, NER, HyDE, cross-encoders, ColBERT, and reranking.
Module 4, PDF pp. 2-96	Transformer and token generation, sampling, model selection, prompt templates, in-context examples, context management, hallucinations, citations, RAGAS measures, agentic workflows, and fine-tuning.
Module 5, PDF pp. 2-73	Production challenges, evaluation and logging, traces, custom datasets, quantization, cost and latency tuning, caching, sharding, security, vector reconstruction risk, and multimodal RAG.

Interview practice questions and model answers are added material. They are not represented as questions taken from the course PDFs.